МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

Львівський національний університет імені Івана Франка

Факультет електроніки та комп’ютерних технологій

ЗВІТ

З ЛАБОРАТОРНОЇ РОБОТИ № 8

з дисципліни “Професійний Python”

на тему “ **Мережеве та асинхронне програмування. REST API** “

Виконав

студент 3 курсу

групи ФЕП-32

Козачок Ростислав Михайлович

Перевірив

Куньо Іван Михайлович

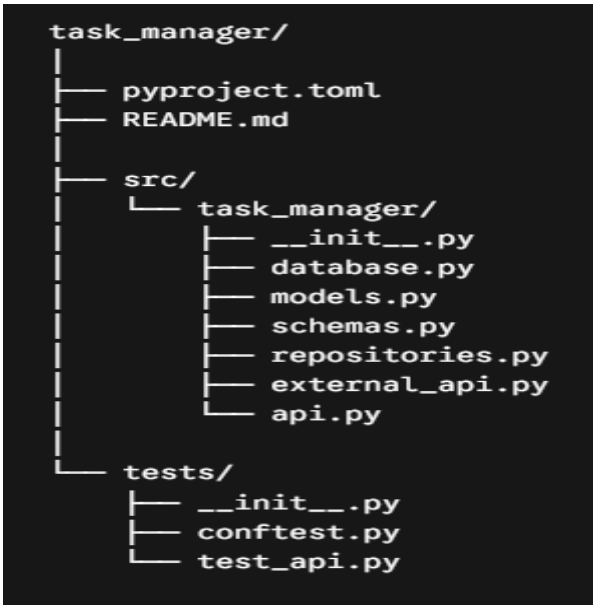
Львів – 2026

**Мета:** Ознайомитися з основами мережевого програмування у Python, принципами роботи протоколу HTTP, архітектурним стилем REST та форматом JSON.
Навчитися створювати REST API за допомогою FastAPI, описувати та перевіряти структуру даних за допомогою Pydantic, використовувати асинхронне програмуванняasync/await та asyncio, виконувати декілька операцій паралельно за допомогою Task і gather(), встановлювати обмеження часу виконання операцій, реалізовувати повторні спроби запитів та обмежувати кількість одночасних операцій за допомогою Semaphore. Навчитися використовувати бібліотеку HTTPX для взаємодії з REST API та зовнішніми HTTP-сервісами.

**Варіант -** 14.

## Хід роботи:

**Структура проєкту**




**Опис REST API та таблиця методів**

| **Метод** | **URL** | **Опис** | **Статус успіху** |
|---|---|---|---|
| GET | /tasks | Отримання списку завдань (із фільтрами) | 200 OK |
| POST | /tasks | Створення нового завдання | 201 Created |
| GET | /tasks/{id} | Деталі завдання за ідентифікатором | 200 OK |

| Метод | URL | Опис | Статус успіху |
|---|---|---|---|
| PATCH | /tasks/{id} | Часткове оновлення завдання | 200 OK |
| DELETE | /tasks/{id} | Видалення завдання | 204 No Content |
| GET | /tasks/overdue | Список прострочених завдань | 200 OK |
| GET | /tasks/statistics | Підрахунок статистики за завданнями | 200 OK |
| GET | /users/{id}/tasks | Завдання конкретного користувача | 200 OK |
| POST | /users | Створення користувача | 201 Created |
| POST | /tasks/batch-users-summary | Асинхронний збір завдань групи користувачів | 200 OK |

**Pydantic-схеми та правила валідаці**

- title: мінімальна довжина 2 символи, максимальна — 200.
- status: обмеження фіксованим списком значень Literal["todo", "in_progress", "done"].
- priority: допустимі значення Literal["low", "medium", "high"].
- email: валідація формату електронної пошти (EmailStr або рядок валідованого формату).
- При передачі некоректних типів даних або значень FastAPI автоматично повертає статус 422 Unprocessable Entity.

**Приклади JSON запитів та відповідей**

**Request Body:**

```
{
  "title": "Налаштувати CI/CD",
```

```
  "description": "Створити конфігурацію GitHub Actions",
  "status": "todo",
  "priority": "high",
  "due_date": "2026-11-01T12:00:00",
  "user_id": 1
}
```


**Response Body (201 Created):**


```
{
  "id": 1,
  "title": "Налаштувати CI/CD",
  "description": "Створити конфігурацію GitHub Actions",
  "status": "todo",
  "priority": "high",
  "due_date": "2026-11-01T12:00:00",
  "user_id": 1
}
```


**Отримання статистики (GET /tasks/statistics):**

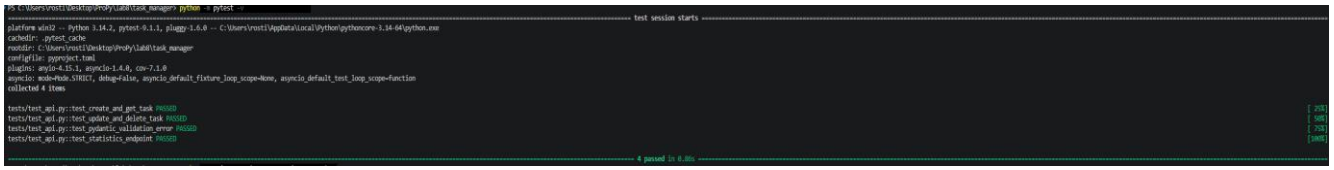
**Response Body (200 OK):**


```
{
  "total": 12,
  "completed": 5,
  "pending": 7,
  "overdue": 2
}
```


**Реалізація асинхронного блоку**

- **async/await**: Усі маршрути FastAPI та клієнтські функції визначені як async def і використовують неблокуючі виклики await client.get(...).
- **asyncio.Task та asyncio.gather()**: Створення списку завдань планувальника через asyncio.create_task() та їх паралельне виконання єдиним пулом через asyncio.gather(*tasks).
- **asyncio.Semaphore**: Обмеження одночасних мережевих підключень до сервісу (пул на максимум 3 одночасні запити).
- **Timeout**: Захист від зависання через asyncio.wait_for(..., timeout=8.0).
- **Retry**: Реалізація повторних запитів (до 3 спроб) при виникненні httpx.RequestError із затримкою asyncio.sleep(0.5 * attempt).

**Результати автоматизованого тестування (pytest)**




**Висновок:**

Під час виконання лабораторної роботи №8 було створено асинхронний REST API для «Системи керування завданнями» (Варіант 14) на базі FastAPI. Було побудовано багаторівневу архітектуру: шар схем Pydantic для валідації, шар бізнес-моделей та репозиторіїв (SQLAlchemy), шар контролерів API. Було практично реалізовано методи асинхронного програмування на базі asyncio та httpx (конкурентні завдання через Task і gather, обмеження навантаження семафором, обробка помилок із retry та тайм-аути). Працездатність системи та коректність статус-кодів підтверджено успішним проходженням модульних та інтеграційних тестів у pytest.